Laboratory work 10

Setting Up the Frontend for Our Spring Boot RESTful Web Service

This chapter explains the steps that are required to start the development of the frontend part of our car database application. We will first define the functionalities that we are developing. Then, we will do a mock-up of the UI. As a backend, we will use our Spring Boot application from *Chapter 5*, *Securing Your Backend*. We will begin development using the unsecured version of the backend. Finally, we will create the React app that we will use in our frontend development.

In this chapter, we will cover the following topics:

- Mocking up the UI
- Preparing the Spring Boot backend
- Creating the React project for the frontend

Technical requirements

The Spring Boot application that we created in *Chapter 5*, *Securing Your Backend*, is required.

Node.js also has to be installed, and the code available at the following GitHub link will be required to follow along with the examples in this chapter: `https://github.com/PacktPublishing/FullStack-Development-with-Spring-Boot-3-andReact-Fourth-Edition/tree/main/Chapter12`.

Mocking up the UI

In the first few chapters of this book, we created a car database backend that provides the RESTful API. Now, it is time to start building the frontend for our application.

We will create a frontend with the following specifications:

- It lists cars from the database in a table and provides **paging**, **sorting**, and **filtering**.
- There is a button that opens a **modal form** to add new cars to the database.
- In each row of the car table, there is a button to **edit** the car or **delete** it from the database.
- There is a link or button to **export** data from the table to a **CSV** file.

UI mock-ups are often created at the beginning of frontend development to provide customers with a visual representation of what the user interface will look like. Mock-ups are quite often done by designers and then provided to developers. There are lots of different applications for creating mock-ups, such as Figma, Balsamiq, and Adobe XD, or you can even use a pencil and paper. You can also create interactive mock-ups to demonstrate a number of functionalities.

If you have done a mock-up, it is much easier to discuss requirements with the client before you start to write any actual code. With the mock-up, it is also easier for the client to understand the idea of the frontend and suggest corrections for it. Changes to mock-ups are really easy and fast to implement, compared to modifications involving actual frontend source code.

The following screenshot shows the example mock-up of our car list frontend:

Figure 12.1: The frontend mock-up

The modal form that is opened when the user presses the + **CREATE** button looks like the following:

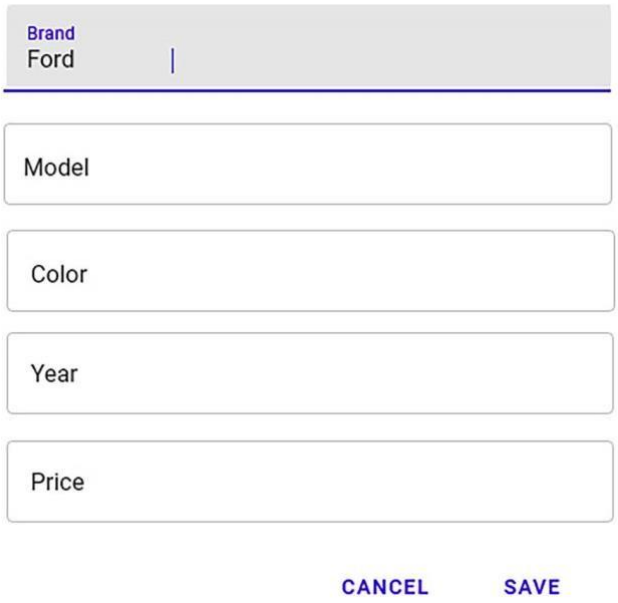

Figure 12.2: The modal form mock-up

Now that we have a mock-up of our UI ready, let's look at how we can prepare our Spring Boot backend.

Preparing the Spring Boot backend

We will begin frontend development in this chapter with the unsecured version of our backend. Then:

- In *Chapter 13*, *Adding CRUD Functionalities*, we will implement all the CRUD functionalities.
- In *Chapter 14*, *Styling the Frontend with MUI*, we will continue to polish our UI using Material UI components.
- Finally, in *Chapter 16*, *Securing Your Application*, we will enable security in our backend, make some modifications that are required, and implement authentication.

In Eclipse, open the Spring Boot application that we created in *Chapter 5*, *Securing Your Backend*. Open the `SecurityConfig.java` file that defines the Spring Security configuration. Temporarily comment out the current configuration and give everyone access to all endpoints. Refer to the following modifications:

```
@Bean public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception
  {
    // Add this one     http.csrf((csrf) ->
csrf.disable()).cors(withDefaults())
                          .authorizeHttpRequests((authorizeHttpRequests) ->
                          authorizeHttpRequests.anyRequest().permitAll());
      /* COMMENT THIS OUT http.csrf((csrf) -> csrf.disable())
.cors(withDefaults())
.sessionManagement((sessionManagement) ->
sessionManagement.sessionCreationPolicy(\
SessionCreationPolicy.STATELESS))
.authorizeHttpRequests( (authorizeHttpRequests) ->
authorizeHttpRequests         .requestMatchers(HttpMethod.POST,
"/login").permitAll()
        .anyRequest().authenticated())
        .addFilterBefore(authenticationFilter,
            UsernamePasswordAuthenticationFilter.class)
.exceptionHandling((exceptionHandling) ->
exceptionHandling.authenticationEntryPoint(
exceptionHandler));
    */
    return http.build();
}
```

Now, if you start the MariaDB database, run the backend, and send the `GET` request to the `http://localhost:8080/api/cars` endpoint, you should get all the cars in the response, as shown in the following screenshot:

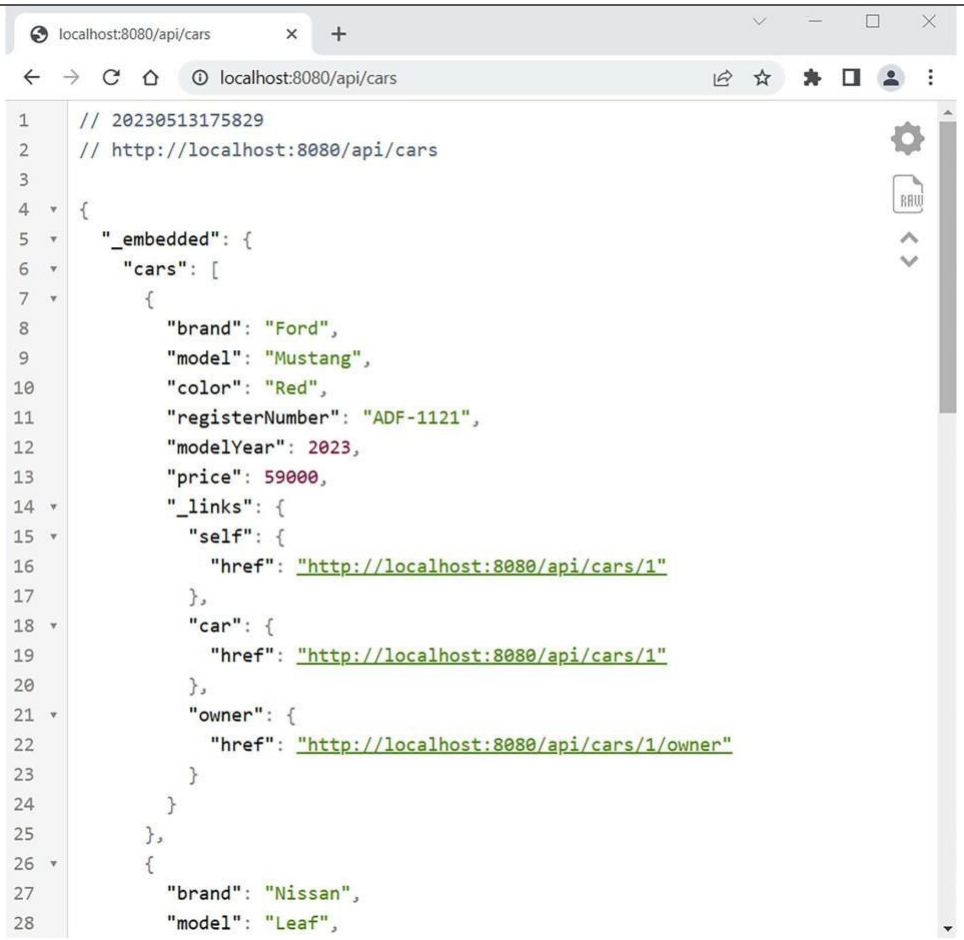

Figure 12.3: The GET request

Now, we are ready to create our React project for the frontend.

Creating the React project for the frontend

Before we start coding the frontend, we have to create a new React app. We will use TypeScript in our React frontend:

1. Open PowerShell, or any other suitable terminal. Create a new React app by typing the following command:

```
npm create vite@4
```

In this book, we are using Vite version 4.4. You can also use the latest version, but then you have to check for changes in the Vite documentation.

2. Name your project `carfront`, and select the **React** framework and **TypeScript** variant:

```
PS C:\> npm create vite@4
√ Project name: ... carfront
√ Select a framework: » React
√ Select a variant: » TypeScript

Scaffolding project in C:\carfront ...

Done. Now run:

  cd carfront
  npm install
  npm run dev
```

Figure 12.4: Frontend project

3. Move to the project folder and install dependencies by typing the following commands:

```
cd carfront
npm install
```

4. Install the MUI component library by typing the following command, which installs the Material UI core library and two Emotion libraries. **Emotion** is a library designed for writing CSS with JavaScript (`https://emotion.sh/docs/introduction`):

```
npm install @mui/material @emotion/react @emotion/styled
```

5. Also, install React Query v4 and Axios, which we will use for networking in our frontend:

```
npm install @tanstack/react-query@4 npm
install axios
```

6. Run the app by typing the following command in the project's `root` folder:

```
npm run dev
```

7. Open the `src` folder with Visual Studio Code and remove any superfluous code from the `App.tsx` file. The file extension is now `.tsx` because we are using TypeScript. Also, remove the `App.css` style sheet file import. We will use the MUI `AppBar` component in the `App.tsx` file to create the toolbar for our app.

We have already looked at the MUI `AppBar` in *Chapter 11*, *Useful Third-Party Components for React* if you would like a reminder.

As shown in the code below, wrap the `AppBar` component inside the MUI `Container` component, which is a basic layout component that centers your app content horizontally. We can use the `position` prop to define the positioning behavior of the app bar. The value `static` means that the app bar is not fixed to the top when the user scrolls. If you use `position= "fixed"`, it will fix the app bar at the top of the page. You also have to import all the MUI components that we are using:

```
import AppBar from '@mui/material/AppBar'; import
Toolbar from '@mui/material/Toolbar'; import
Typography from '@mui/material/Typography'; import
Container from '@mui/material/Container'; import
CssBaseline from '@mui/material/CssBaseline';

function App() {   return
(
    <Container maxWidth="xl">
    <CssBaseline />
      <AppBar position="static">
<Toolbar>
          <Typography variant="h6">
            Car Shop
          </Typography>
        </Toolbar>
</AppBar>
    </Container>
  );
}

export default App;
```

The `maxWidth` prop defines the maximum width of our app, and we have used the largest value. We have also used the MUI `CssBaseline` component, which is used to fix inconsistencies across browsers, ensuring that the React app's appearance is uniform across different browsers. It is typically included at the top level of your application to ensure that its styles are applied globally.

8. We will remove all predefined styling. Therefore, remove the `index.css` style sheet import from the `main.tsx` file. The code should look like the following:

```
import React from 'react' import ReactDOM
from 'react-dom/client' import
App from './App.tsx'

ReactDOM.createRoot(document.getElementById('root') as HTMLElement).
render(
  <React.StrictMode>
    <App />
</React.StrictMode>,
)
```

Now, your frontend starting point should look like this:

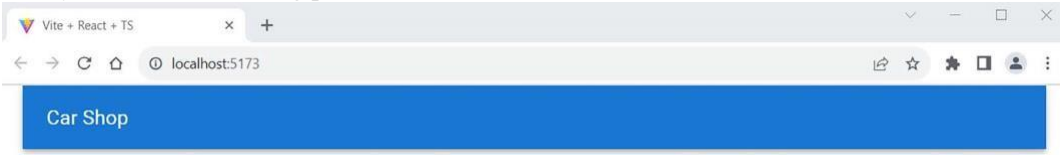

Figure 12.5: Car Shop

We have now created the React project for our frontend and can continue with further development.